

灵敏度

Contents

- 8.1. 什么叫灵敏度
- 8.2. Sobol 指标
- 8.3. 计算方法
- 8.4. 总结

8.1. 什么叫灵敏度

灵敏度分析是研究与分析一个系统（数学模型）输入参数的变动对其输出结果的影响程度的方法。

对于一个函数：

$$y = f(x_1, x_2, \dots, x_n)$$

如果输入参数 x_1 变化了 Δx_1 ，那么输出 y 的变化量 Δy 可以表示为：

$$\Delta y = \frac{\partial f}{\partial x_1} \Delta x_1$$

这个表征的是在某个局部点，输入参数 x_1 变化了 Δx_1 ，输出 y 的变化量 Δy 。这也成为**局部灵敏度**。

与之对应，对一个不能抽象为无穷小的有限区域中，输入参数的变化对输出参数的变化的影响，称之为**全局灵敏度**。全局灵敏度有很多表示方法，最为常见的是 **Sobol 指标**。

8.2. Sobol 指标

Sobol指标是基于variance（方差）的指标。对于前面的函数，把输入的参数

$$X = (x_1, x_2, \dots, x_n)$$

当做一个随机向量，根据输入变量的方差和输出变量的方差来描述输入变量对输出变量的贡献程度。

因此上面的函数写为随机变量的形式：

$$Y = f(X)$$

这里实际上核心要处理的问题就是 n 为变量之间的组合和相关性的问题。对于 $1, \dots, n$ 的任意子集合 $I \subseteq \{1, \dots, n\}$ ，定义：

$$X_I = (x_i)_{i \in I}$$

可以把 Y 的方差表达为 X 的函数。

$$\text{Var}(Y) = \sum_{I \subseteq \{1, \dots, n\}} V_I \quad (1)$$

这里的 V_I 表示的是 I 的所有子集合给 Y 带来的方差。

$$V_I = \text{Var} \left[\sum_{J \subseteq I} (-1)^{|I|-|J|} \mathbb{E}[Y|X_J] \right] \quad (2)$$

这个公式非常抽象，但是理解下意思就行了。我们可以从1、2个变量的集合来理解。 $\hat{V}_i$ 表示的是变量 x_i 对 Y 的贡献， $\hat{V}_{ij}$ 表示的是变量 x_i 和 x_j 对 Y 的联合贡献。

对于 $1 \leq i, j \leq n$ ，定义 $\hat{V}_i = V_{\{i\}}$ ， $\hat{V}_{ij} = V_{\{i,j\}}$ 。

$$\hat{V}_i = \text{Var} [\mathbb{E}[Y|X_i]]$$

$$\begin{aligned} \hat{V}_{ij} &= \text{Var} [\mathbb{E}[Y|X_i, X_j]] - \mathbb{E}[Y|X_i] - \mathbb{E}[Y|X_j] \\ &= \text{Var} [\mathbb{E}[Y|X_i, X_j]] - \hat{V}_i - \hat{V}_j \end{aligned}$$

所以 Y 的方差可以写为如下：

$$\text{Var}(Y) = \sum_{i=1}^n \hat{V}_i + \sum_{1 \leq i < j \leq n} \hat{V}_{ij} + \dots + \hat{V}_{1,2,\dots,n} \quad (3)$$

因为有式(1)，可以把式(3)写为：

$$\sum_{i=1}^n S_i + \sum_{1 \leq i < j \leq n} S_{i,j} + \cdots + S_{1,2,\dots,n} = 1 \quad (4)$$

这里的 $S_i = \hat{V}_i / \text{Var}(Y)$ 表示的是变量 x_i 对 Y 的贡献（**一阶Sobol指标**）， $S_{i,j} = \hat{V}_{ij} / \text{Var}(Y)$ 表示的是变量 x_i 和 x_j 对 Y 的联合贡献。

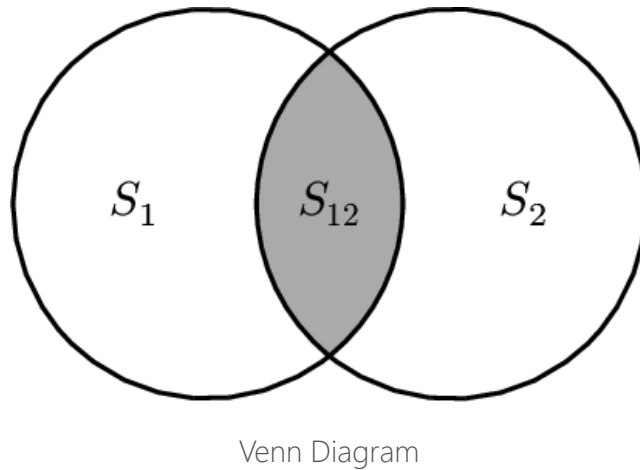
在这些概念的基础上，可以定义**全阶Sobol指标**（Total Effect Index），首先定义

$$VT_i = \sum_{I, i \in I} V_I, \quad V_{-i} = \text{Var} [\mathbb{E}[Y | X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_n]] \quad (5)$$

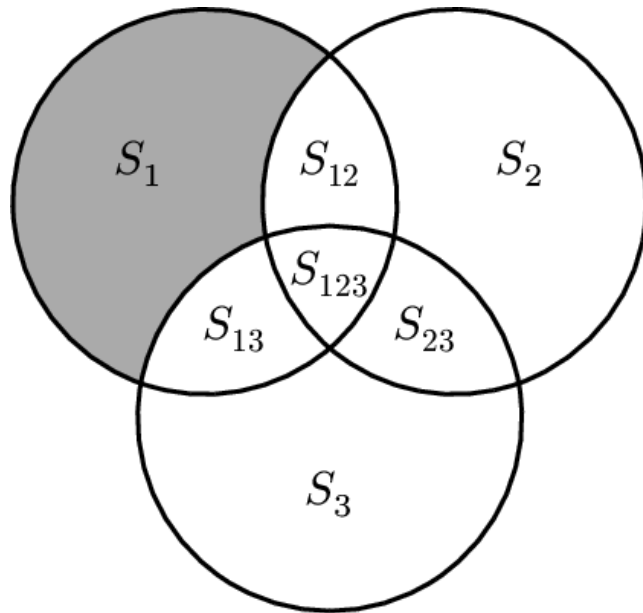
这里 VT_i 是对所有包含 x_i 的子集合的 V_I 的求和， V_{-i} 是对所有不包含 x_i 的子集合的 V_I 的求和。在此基础上定义，全阶Sobol指标：

$$ST_i = VT_i / \text{Var}(Y) = 1 - V_{-i} / \text{Var}(Y) \quad (6)$$

下面的两个图就显示了 S_i 的含义，也就是 x_i 对 Y 的单独贡献，排除了其他变量的影响。对于两个变量的情况，非常容易理解。

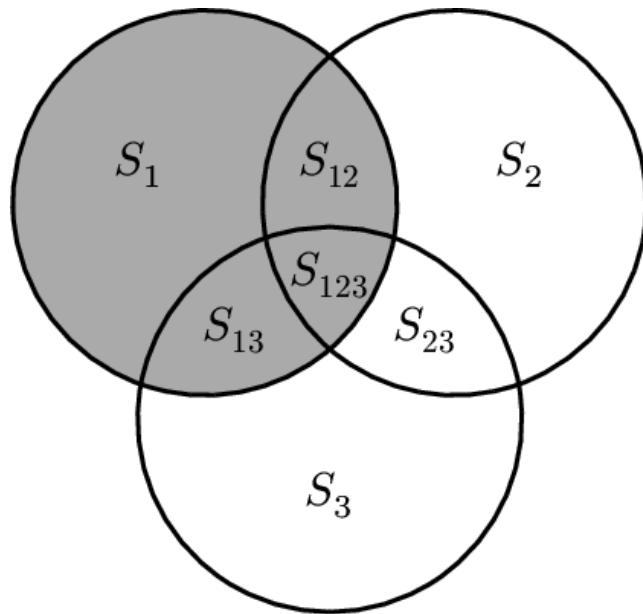


三个变量的情况，也很类似，就是相互交叉的影响一下子增加了很多。



Main effect of variable 1

而下图就显示了 ST_i 的含义。也就是 x_i 对 Y 的全部贡献，包含了其他变量的交互影响。



Total effect of variable 1

$$ST_1 = 1 - S_2 - S_3 - S_{2,3} = S_1 + S_{1,2} + S_{1,3} + S_{1,2,3}$$

8.3. 计算方法

理论很复杂，但计算可以通过Monte Carlo方法实现。我们先从一个最简单的例子开始：

$$y = x_1 + x_2 + x_3, \quad x_1, x_2, x_3 \in [0, 1]$$

凭直觉可以猜到，三个变量的贡献是均等的，且没有交互项，所以：

$$S_1 = S_2 = S_3 = \frac{1}{3}, \quad ST_1 = ST_2 = ST_3 = \frac{1}{3}$$

$$S_{1,2} = S_{1,3} = S_{2,3} = S_{1,2,3} = 0$$

如果我们采用QMC（Quasi-Monte Carlo）方法，应该怎么计算呢？

首先，均值和方差可以通过采样计算：

$$\bar{y} \approx \frac{1}{N} \sum_{i=1}^N f(x_i)$$

$$\text{Var}(y) \approx \frac{1}{N} \sum_{i=1}^N f(x_i)^2 - \bar{y}^2$$

对应的Sobol指标计算实际上要做两次采样。我们生成两个独立的 $N \times D$ 的样本矩阵 A 和 B （ D 是变量个数）。

为了计算 S_i ，我们需要一个特殊的矩阵 C_i ，它由矩阵 A 和 B 构成： C_i 的所有列来自矩阵 A ，只有第 i 列来自矩阵 B 。

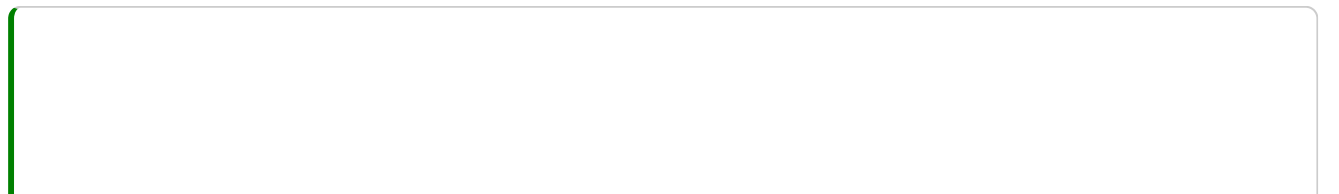
根据Saltelli (2010) 的方法，一阶和全阶指标可以如下估算：

$$S_i = \frac{\frac{1}{N} \sum_{j=1}^N f(B)_j (f(C_i)_j - f(A)_j)}{\text{Var}(Y)}$$

$$ST_i = \frac{\frac{1}{2N} \sum_{j=1}^N (f(A)_j - f(C_i)_j)^2}{\text{Var}(Y)}$$

这里的 $f(A)_j$ 表示用矩阵 A 的第 j 行作为输入得到的函数值。

下面我们用Python代码来实现这个过程。



```

import numpy as np
from scipy.stats import qmc

# 1. 定义模型函数
def model(x):
    """一个简单的线性模型"""
    return np.sum(x, axis=1)

# 2. 定义问题参数
problem = {
    'num_vars': 3,
    'names': ['x1', 'x2', 'x3'],
    'bounds': [[0, 1], [0, 1], [0, 1]]
}

# 3. 生成样本
# 使用 Sobol 序列生成更均匀的样本
sampler = qmc.Sobol(d=2 * problem['num_vars'], scramble=True)
N = 1024 # 样本数量
samples = sampler.random_base2(m=10) #  $2^{10} = 1024$ 

# 将样本缩放到定义的边界
l_bounds = [b[0] for b in problem['bounds']]
u_bounds = [b[1] for b in problem['bounds']]
scaled_samples = qmc.scale(samples, l_bounds * 2, u_bounds * 2)

# 划分样本矩阵 A 和 B
A = scaled_samples[:, :problem['num_vars']]
B = scaled_samples[:, problem['num_vars']:]

# 4. 运行模型
y_A = model(A)
y_B = model(B)

# 计算总方差
var_Y = np.var(y_A)

# 5. 计算 Sobol 指标
S = []
ST = []

for i in range(problem['num_vars']):
    # 创建矩阵 C_i
    C_i = np.copy(A)
    C_i[:, i] = B[:, i]

    # 运行模型得到 y_C_i
    y_C_i = model(C_i)

    # 计算一阶指标 S_i
    s_i = np.mean(y_B * (y_C_i - y_A)) / var_Y
    S.append(s_i)

    # 计算全阶指标 ST_i
    st_i = 0.5 * np.mean((y_A - y_C_i)**2) / var_Y
    ST.append(st_i)

print("一阶 Sobol 指标 (S_i):")

```

```
for i, s_i in enumerate(S):
    print(f" {problem['names'][i]}: {s_i:.4f}")

print("\n全阶 Sobol 指标 (ST_i):")
for i, st_i in enumerate(ST):
    print(f" {problem['names'][i]}: {st_i:.4f}")

# 理论值
print("\n理论值:")
print(" S_i = 1/3 ≈ 0.3333")
print(" ST_i = 1/3 ≈ 0.3333")
```

一阶 Sobol 指标 (S_i):

x1: 0.3333
x2: 0.3333
x3: 0.3333

全阶 Sobol 指标 (ST_i):

x1: 0.3333
x2: 0.3333
x3: 0.3333

理论值:

S_i = 1/3 ≈ 0.3333
ST_i = 1/3 ≈ 0.3333

8.4. 总结

从上面的计算结果可以看出，通过Monte Carlo方法计算出的Sobol指标与理论值非常接近，验证了该方法的有效性。

对于更复杂的非线性模型，这种方法同样适用，是进行全局灵敏度分析的有力工具。